



JSPM University Pune
Faculty of Science and Technology
School of Computational Sciences
TY BTECH CSE

Experiment No: 1

Date: / /

Aim : Introduction to virtual platform Wokwi (<https://wokwi.com/>)

OBJECTIVE: The main objective of this experiment is,

- A. Create an account on Wokwi
- B. Identify the different devices and measuring instrument.
- C. Simulate a simple circuit in Wokwi

DESCRIPTION:

Wokwi is a browser-based online electronics and IoT simulator that allows users to build, code, and test embedded systems projects in a virtual environment. It supports popular boards like Arduino, ESP32, STM32, and Raspberry Pi Pico, along with a wide range of virtual components.

Key features:

- Browser-based: No installations or downloads are required, allowing users to start simulating directly from any modern web browser.
- Virtual hardware: Users can add and wire components virtually, such as LEDs, sensors, displays, and motors, and interact with them in real-time.
- Code editor: A built-in code editor lets you write and edit code directly in the platform, with support for languages like C++ (for Arduino) and MicroPython.
- IoT capabilities: The simulator includes virtual Wi-Fi and networking features, allowing you to test internet-connected projects using protocols like MQTT and HTTP.
- VS Code integration: A Visual Studio Code extension is available, enabling embedded and IoT developers to integrate Wokwi's simulation directly into their local development workflow.
- Debugging tools: Advanced tools like a virtual logic analyzer and GDB integration help in debugging code and analyzing digital signals.
- Collaboration: Projects can be easily shared with others via a simple link, which is useful for remote learning and team projects.

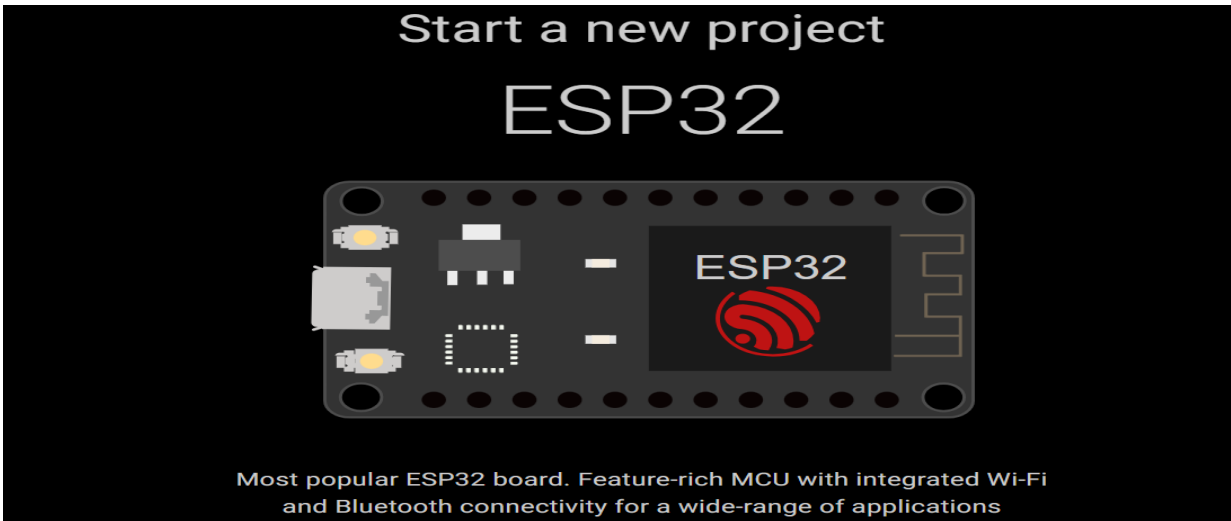
Common uses:

- **Education:** It serves as a teaching tool for students to learn about microcontrollers, electronics, and programming without needing physical hardware.
- **Prototyping:** Developers and hobbyists can quickly test and iterate on project ideas, saving time and money on components.
- **Debugging:** Users can diagnose issues with code and circuit design in a virtual environment before moving to physical hardware.
- **Development:** It is used for developing and testing firmware for embedded and IoT applications.

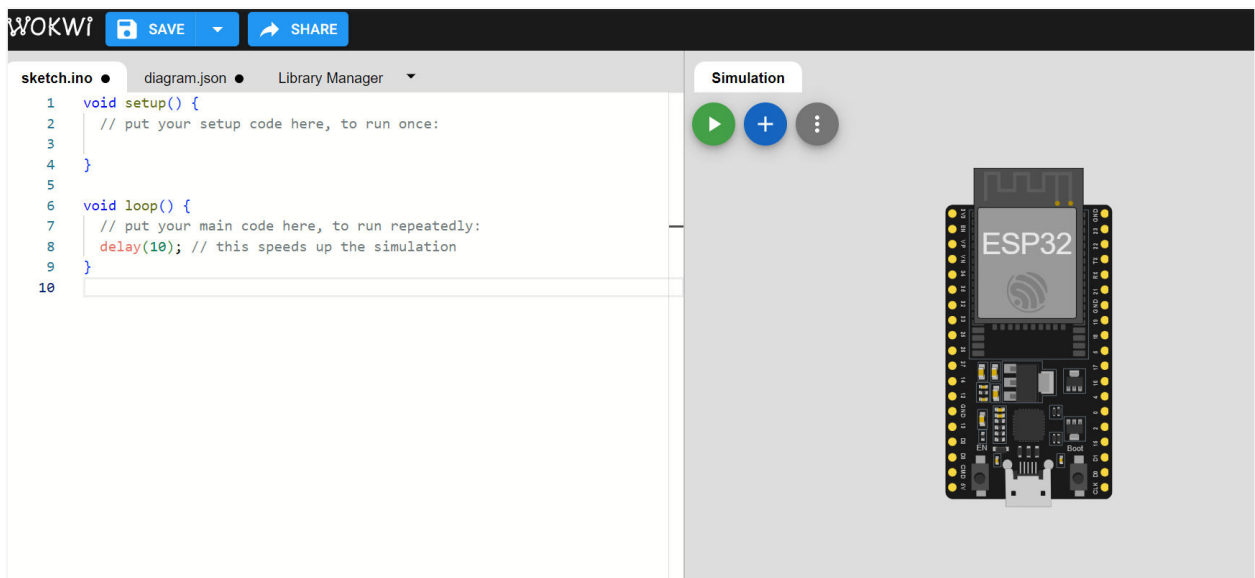
STEPS TO CREATE NEW PROJECT:

To get started, navigate to the Wokwi Simulator website. From there, you'll see the following dashboard. You can use the app for free and without creating a user account.

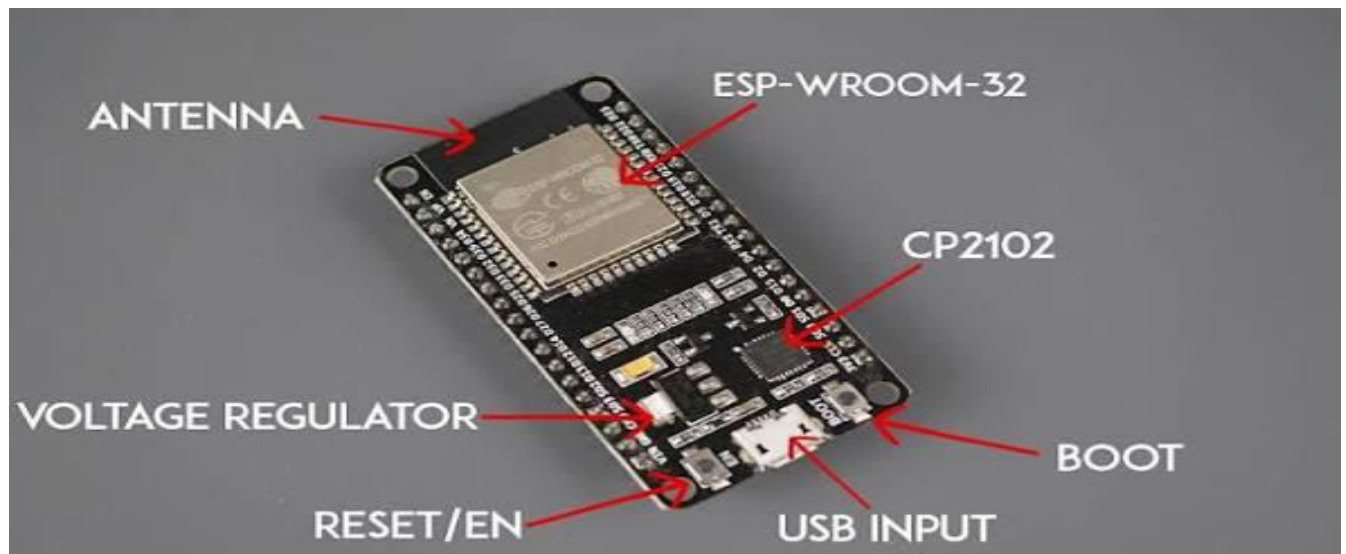




Wokwi Main Sheet

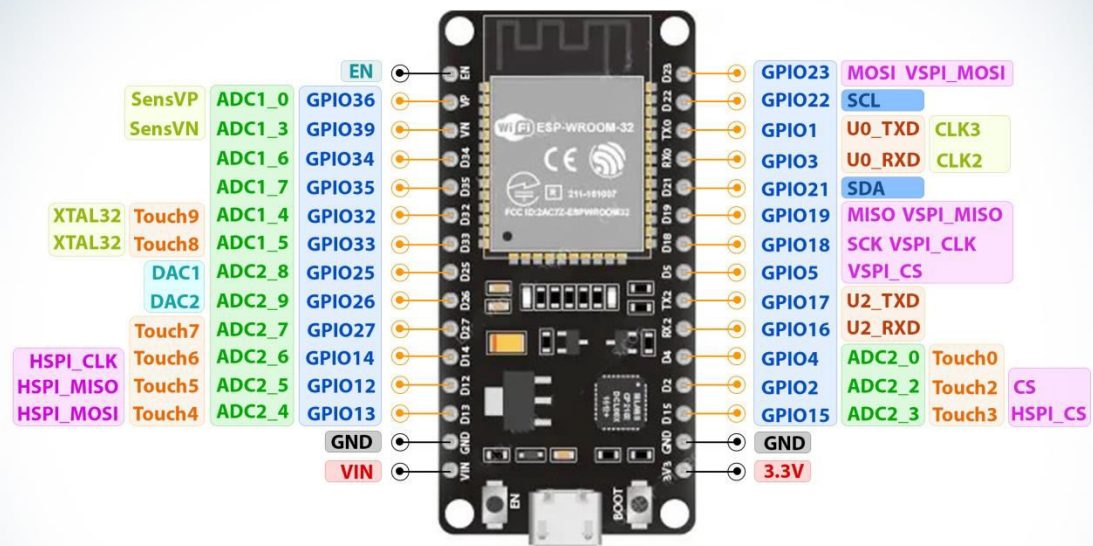


The ESP32 is a powerful and versatile system-on-a-chip (SoC) microcontroller, known for its integrated Wi-Fi and Bluetooth capabilities. This makes it a popular choice for Internet of Things (IoT) applications, embedded systems, and other projects requiring wireless connectivity.



ESP32 PIN Diagram :-

ESP32 DEV. BOARD PINOUT



Key features:

Processing and memory

- Dual-core or single-core processor: The standard ESP32 features a dual-core, 32-bit Tensilica Xtensa LX6 microprocessor, with some variants like the [ESP32-S0WD](#) using a single-core version. Newer ESP32-C series models use a single-core RISC-V CPU, and [ESP32-S3](#) uses a dual-core Xtensa LX7 CPU with machine learning acceleration.
- Adjustable clock frequency: The clock speed is adjustable from 80 MHz to 240 MHz, allowing developers to balance performance and power consumption.
- Ultra-low-power (ULP) co-processor: For extended battery life, the ULP co-processor can perform simple tasks like sensor monitoring while the main CPU is in deep sleep.
- Internal memory: Memory includes 520 KB of on-chip SRAM for data and instructions, plus additional ROM and RTC SRAM for boot and low-power operations.
- Expandable flash memory: Modules typically come with 4 MB of external flash memory for storing programs and data, with some variants supporting additional external PSRAM.

Wireless connectivity

- Wi-Fi: Supports 802.11 b/g/n Wi-Fi, allowing the ESP32 to connect to routers in Station mode, create its own network in Access Point (SoftAP) mode, or operate in both simultaneously.
- Bluetooth: Features dual-mode Bluetooth connectivity with support for both Classic Bluetooth (BR/EDR) and Bluetooth Low Energy (BLE), enabling it to communicate with a wide range of devices.
- Wi-Fi 6 (802.11ax): Newer variants, such as the ESP32-C6, also feature Wi-Fi 6 support.

Power management

- Multiple low-power modes: The [ESP32](#) is designed for energy efficiency, offering several power-saving modes to optimize battery life.
- Modem-sleep: Wi-Fi and Bluetooth are disabled, but the CPU remains active. Power consumption is ~3–20 mA.

- Light-sleep: The CPU and most RAM are clock-gated, but the Wi-Fi/Bluetooth connection is maintained. Current consumption is approximately 0.8 mA.
- Deep-sleep: Powers down the CPU and most RAM, with only the real-time clock (RTC) and ULP co-processor remaining active. This dramatically reduces power consumption, making it ideal for battery-operated devices.
- Hibernation: The most extreme power-saving mode, where nearly all hardware is turned off, with consumption as low as 5 μ A.

Peripherals and interfaces

- Extensive GPIOs: Offers numerous programmable General-Purpose Input/Output (GPIO) pins for connecting to a wide variety of sensors and peripherals.
- Analog-to-Digital Converter (ADC): Features multiple 12-bit SAR ADC channels for reading analog sensor data.
- Digital-to-Analog Converter (DAC): Includes 8-bit DAC channels for converting digital signals to analog voltages.
- Capacitive touch sensors: Contains built-in touch sensors, which can be used to create touch-sensitive pads without external hardware.
- Common communication protocols: Supports standard interfaces such as SPI, I²C, I²S, UART, and PWM, allowing for flexible connectivity.
- Other peripherals: Depending on the model, additional peripherals can include a Hall sensor, Ethernet MAC interface, and an SD/SDIO/MMC Host controller.

Security

- Secure boot: Ensures that only trusted and authenticated software can execute on the device, preventing untrustworthy code from running.
- Flash encryption: Encrypts the contents of the external flash memory, protecting intellectual property and sensitive data.
- Hardware cryptographic acceleration: Features hardware acceleration for cryptographic algorithms like AES, SHA-2, and RSA, ensuring secure communication.
- Secure OTA updates: Allows for secure Over-the-Air firmware updates, ensuring that new code is authenticated and encrypted during transit.

- RESULTS:

- CONCLUSION:

Sample Viva Questions

1) What is the Wokwi simulator, and how does it work for simulating microcontroller projects?

2) What are the key differences between simulating a project on Wokwi versus testing on actual hardware (e.g., ESP32 or Arduino)?



JSPM University Pune
Faculty of Science and Technology
School of Computational Sciences
TY BTECH CSE

Experiment No: 2

Date: / /

Aim : Arduino IDE Software Installation and Programming Concepts for ESP32

OBJECTIVE: The main objective of this experiment is,

- A. Install Arduino.
- B. Study Programming concept.

Description:

INSTALL ARDUINO IDE

The Arduino IDE (Integrated Development Environment) in reference to the ESP32 is a software platform that allows users to write, compile, and upload code (sketches) to ESP32 development boards. While the ESP32 is a powerful microcontroller with its own official Espressif SDK (ESP-IDF), the Arduino IDE offers a familiar and user-friendly environment for programming the ESP32, especially for those accustomed to the Arduino ecosystem.

Arduino IDE installation steps:

STEP 1: Download Arduino IDE: Navigate to the official Arduino website and download the appropriate version of the Arduino IDE for your operating system (Windows, macOS, Linux).

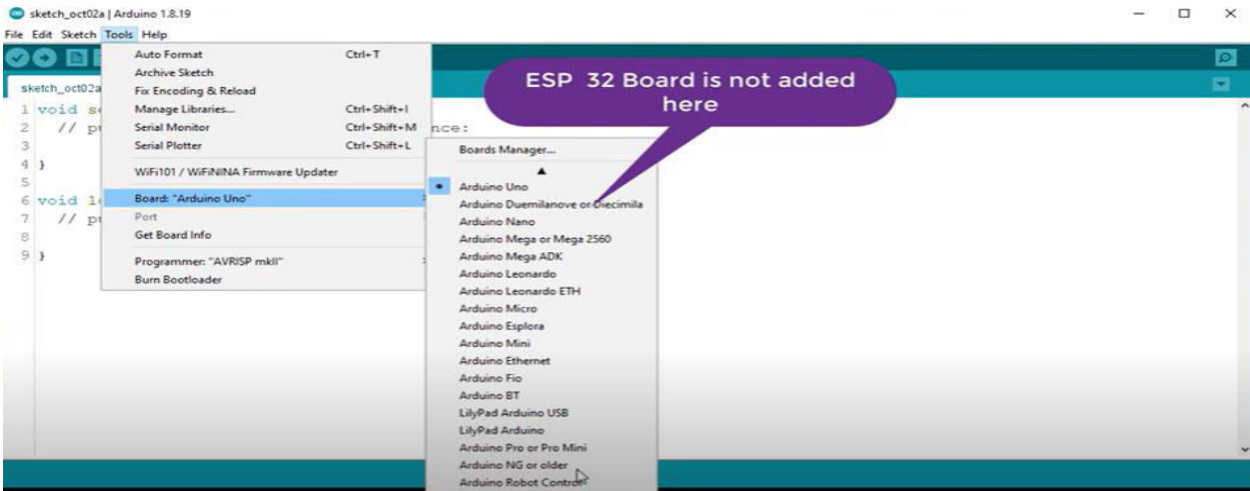
STEP 2: Install Arduino IDE:

Windows: Run the downloaded executable file. Follow the on-screen prompts, agreeing to the license terms and selecting installation options (e.g., install USB driver).

macOS: Drag the downloaded Arduino application file into your Applications folder.

Linux: Extract the downloaded archive and run the arduino-ide executable within the extracted folder.

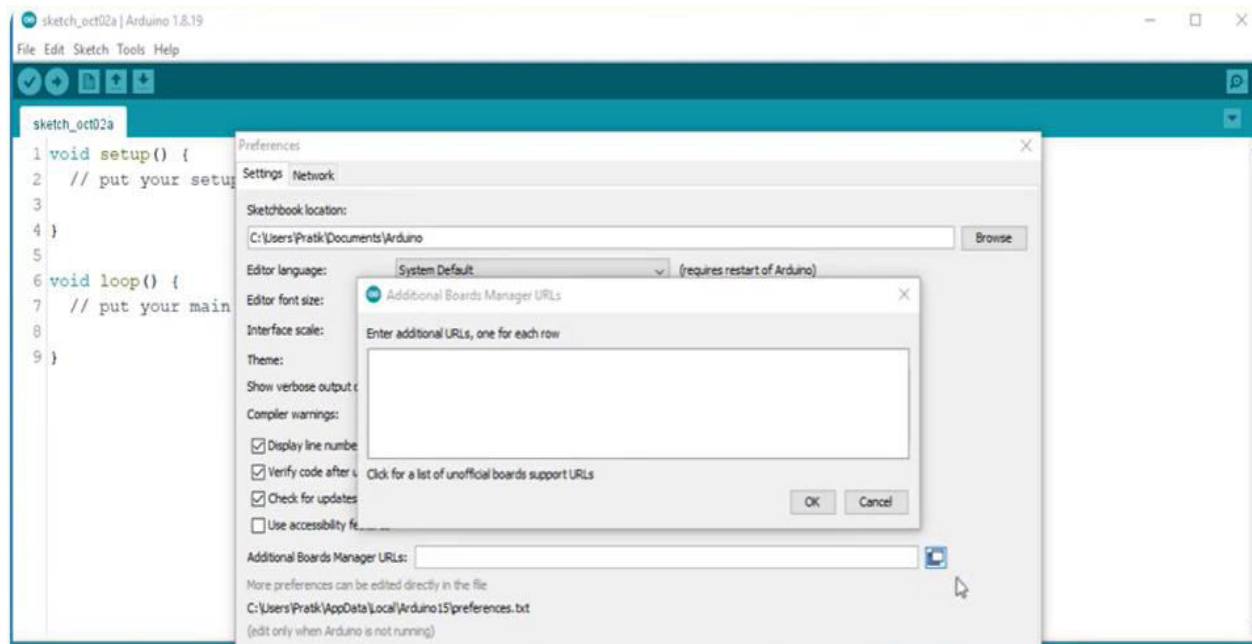
STEP 3: Launch Arduino IDE: Open the Arduino IDE application after the installation is complete.



Installing ESP32 Board Support in Arduino IDE:

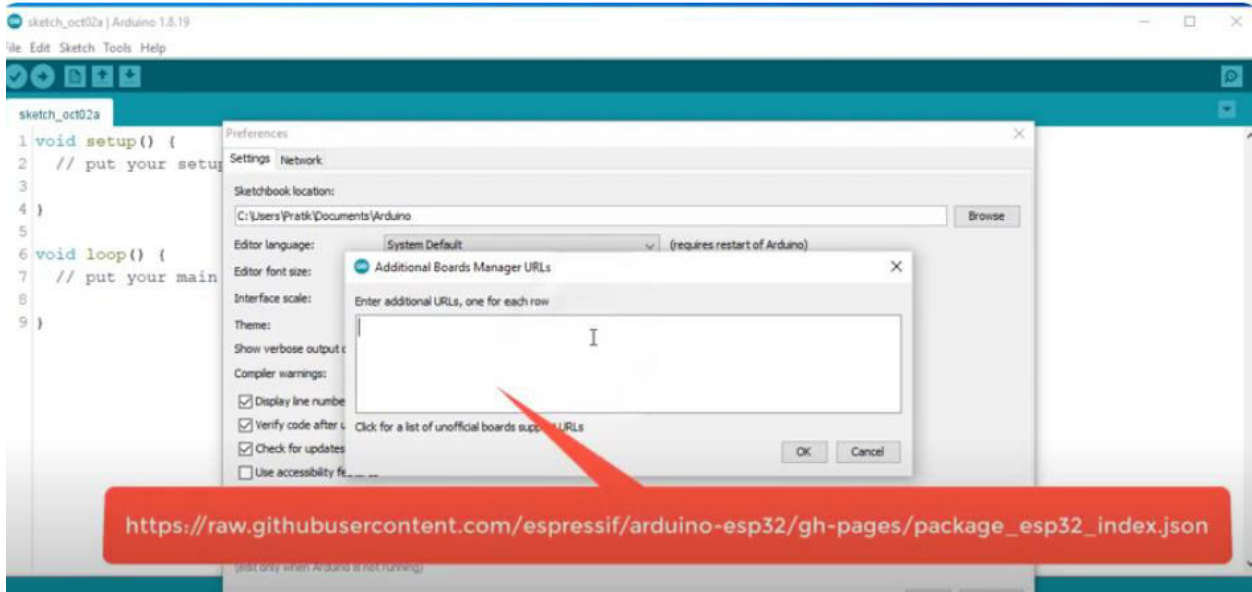
- Add ESP32 Board Manager URL:

STEP 1: In the Arduino IDE, go to File > Preferences.



STEP 2: In the "Additional Boards Manager URLs" field, paste the following URL:

https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json

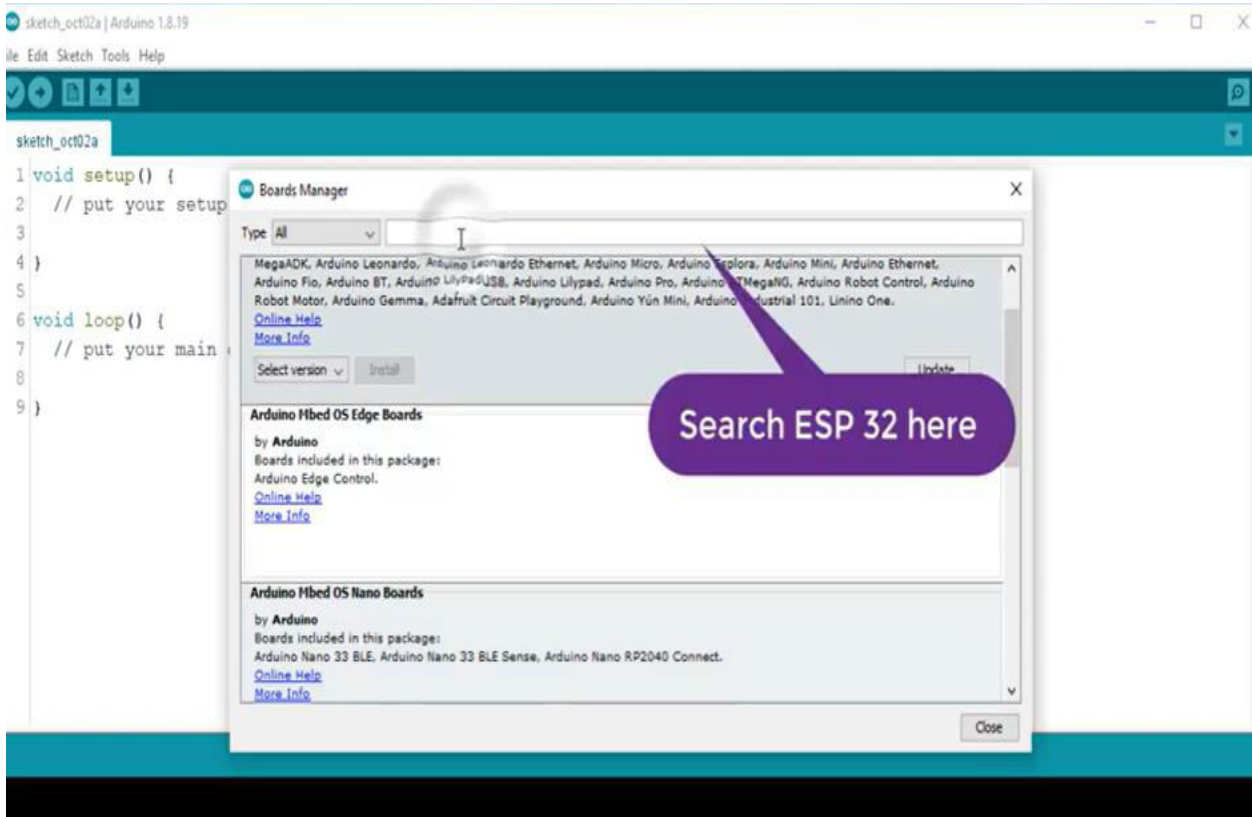


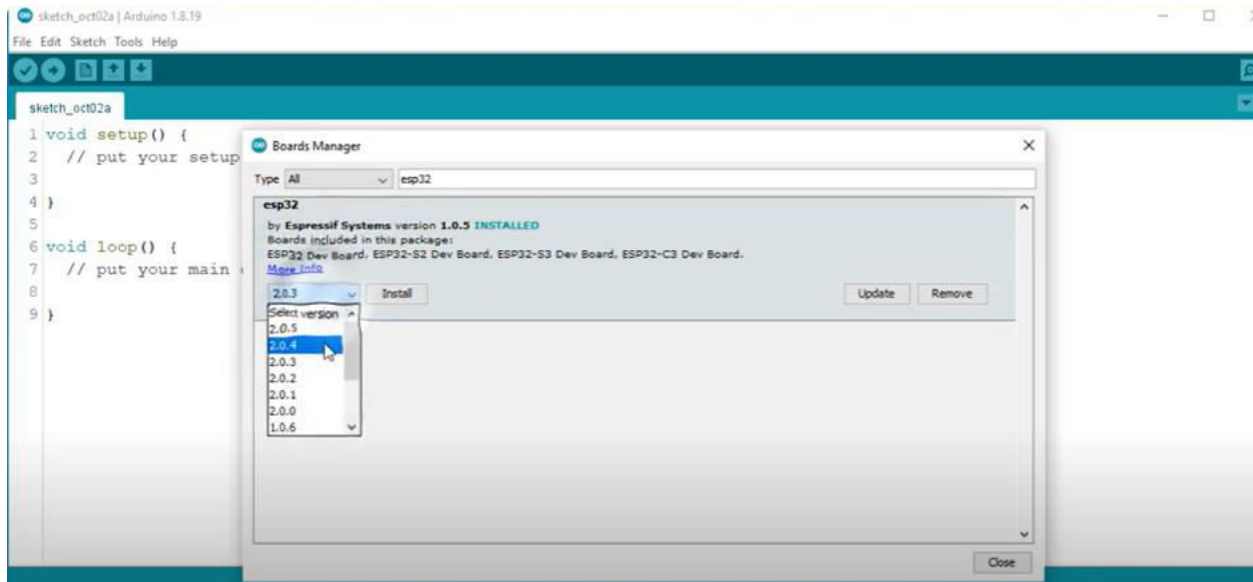
STEP 3: Click "OK".

Note: If you already have other URLs (e.g., ESP8266), separate them with a comma.

- **Open Boards Manager:**

Go to Tools > Board > Boards Manager....



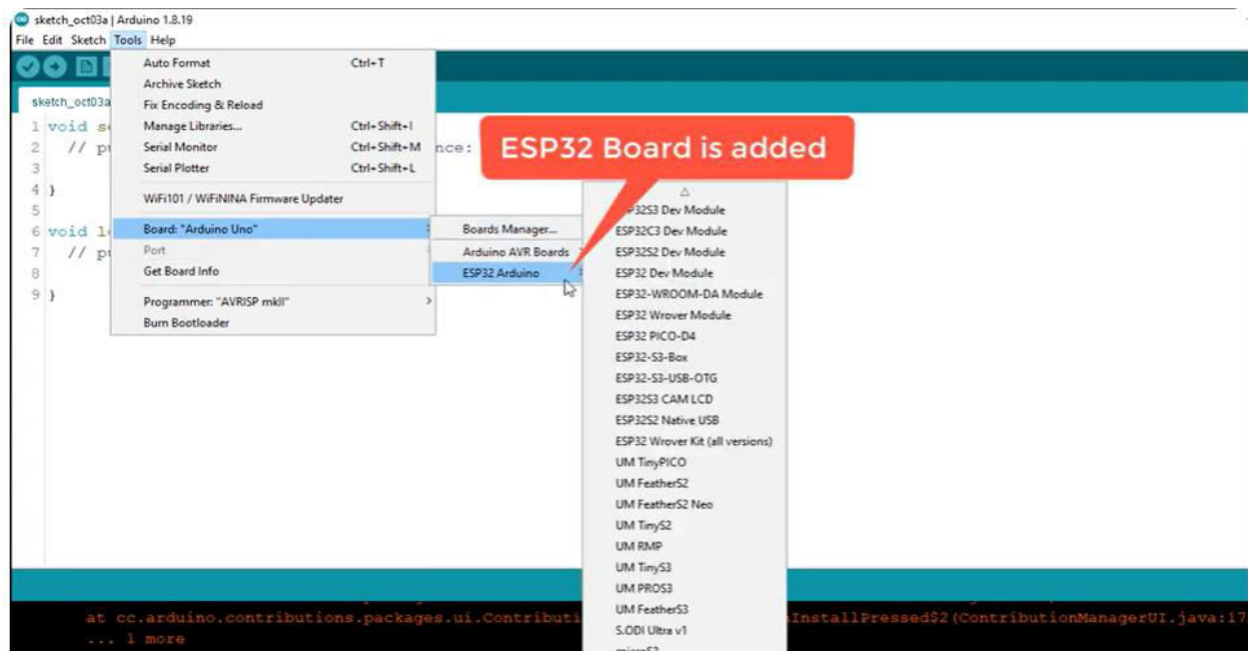


Install ESP32 Boards:

- In the Boards Manager search bar, type "ESP32".
- Locate "ESP32 by Espressif Systems" and click the "Install" button.

Select ESP32 Board:

After the installation completes, go to Tools > Board and you should now see a list of ESP32 boards. Select the specific ESP32 board you are using (e.g., "ESP32 Dev Module", "ESP32 WROOM-32").



- **Select Serial Port:**

Connect your ESP32 board to your computer via a USB cable. Go to Tools > Port and select the COM port corresponding to your connected ESP32 board. (You may need to check your system's Device Manager or equivalent to identify the correct port).

Uploading a Sketch to ESP32:

- **Open or Write a Sketch:**

Open an existing ESP32 example sketch (e.g., File > Examples > ESP32 > Blink) or write your own code.

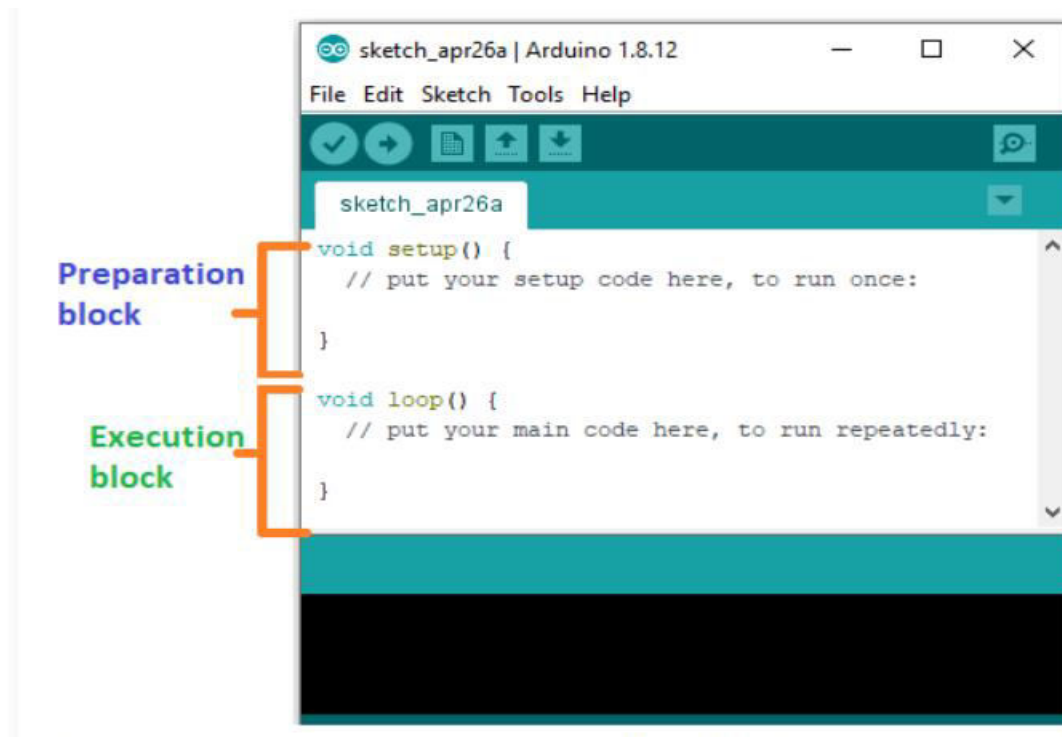
- **Upload:**

Click the "Upload" button (right arrow icon) in the Arduino IDE. You might need to press and hold the "BOOT" button on your ESP32 board during the upload process, as indicated by the IDE's status messages. Release the "BOOT" button once the upload starts.

- **Monitor Output (Optional):**

Open the Serial Monitor (Tools > Serial Monitor) to view any serial output from your ESP32.

BASIC PROGRAMMING CONCEPTS OF ARDUINO IDE:



1. pinMode()

- Purpose: Configures a specific pin to behave either as an input or output.
- Syntax:
`pinMode(pin, MODE);`
MODE can be INPUT, OUTPUT, INPUT_PULLUP
- Example:
`pinMode(13,OUTPUT);`
- Usage: Always set the pin mode in the setup() function before reading from or writing to a pin.

2. digitalWrite() and digitalRead()

- digitalWrite(): Sets a digital pin to HIGH or LOW.
`digitalWrite(pin, HIGH);`
`digitalWrite(pin, LOW);`
- digitalRead(): Reads the state of a digital pin (HIGH or LOW).
`int buttonState = digitalRead(7);`
- Usage:
- Use digitalWrite for controlling LEDs, relays, or other devices.
- Use digitalRead for reading switches or sensors that provide binary signals.

3. analogRead()

- Purpose: Reads the analog voltage (0 to 5V or 0 to 3.3V depending on board) and converts it to a number between 0 and 1023.
- Syntax:
`int sensorValue = analogRead(A0);`
- Usage: Typical for sensors like potentiometers or temperature sensors.

4. analogWrite()

- Purpose: Writes a PWM (Pulse Width Modulation) signal to a pin, simulating analog output.
- Syntax:
`analogWrite(pin, Value);`
- Usage: Controlling LED brightness or motor speed.

5. delay()

- Purpose: Pauses program execution for a specified number of milliseconds.
- Syntax:
`Delay(1000);`
- Usage: Simple timing between actions.

- RESULTS:

- CONCLUSION:

Assignment Question

- 1) What are the main features of the ESP32, and how does it differ from other microcontrollers like the Arduino Uno or NodeMCU?
- 2) How do you set up and upload a simple program, like blinking an LED, to the ESP32 using the Arduino IDE?
- 3) How can you connect the ESP32 to a Wi-Fi network?
- 4) What is the role of the GPIO pins on the ESP32, and how can you use them to control sensors or actuators in a project?



JSPM University Pune
Faculty of Science and Technology
School of Computational Sciences
TY BTECH CSE

Experiment No: 3

Date: / /

Aim: LED blinking using ESP32 (Digital Write).

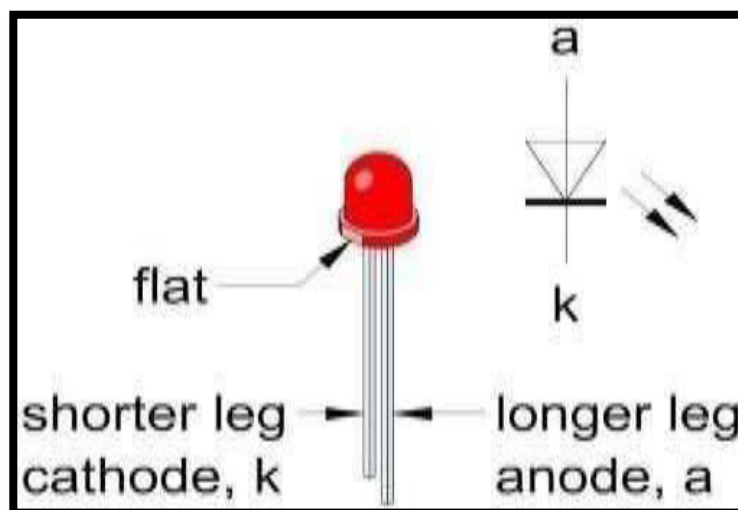
OBJECTIVE: The main objective of this experiment is,

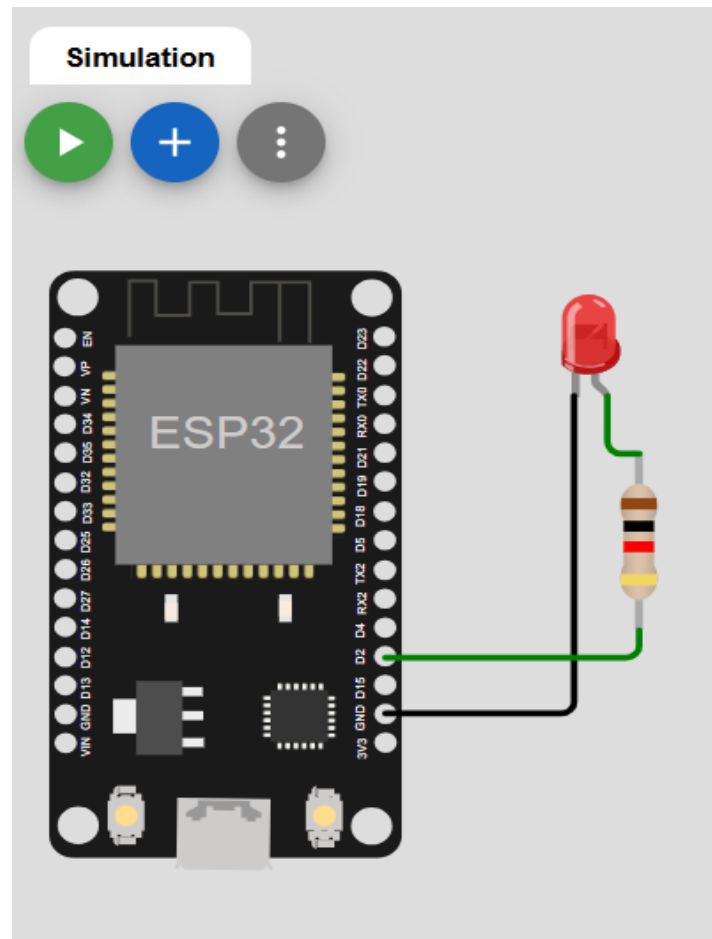
A. To blink an LED with a delay of 2 seconds.

APPARATUS: Arduino UNO Board, LED, Jumper wires, Breadboard, ESP32 .

Description:

LED blinking is the most beginner & easy step to start experiments with arduino. Firstly identify anode (+ve) & cathode (-ve) leg of LED. Following diagram gives a clear idea of LED. Now connects a resistor to positive leg and other end of resistor to 5V. Cathode will be connected to Arduino GPIO, in this case pin 13. Final circuit will look as below:





PROCEDURE:

- Give the connections to the ESP32 board as shown in the connection diagram.
- Open Arduino IDE in the computer
- Create new file → New
- Type your program and **Save** it in appropriate location in your computer.
- Compile your program by clicking **Verify** option in the menu.
- Once the program compiled successfully, connect the Arduino board to the computer using USB cable. After connecting, go to **Tools** → **Board** → Select **Arduino AVR Boards** option
- After selecting board, go to **Tools** → **Port** → Select **Arduino Uno COM port 3**

PROGRAM:

A large, empty rectangular box with a thin black border, intended for the user to provide details about the program. It occupies the majority of the page area below the 'PROGRAM:' label.

- **RESULTS:**

- **CONCLUSION:**

SAMPLE VIVA-VOCE QUESTIONS:

- 1) What are the components required for blinking LED using ESP32?
- 2) What is the Use of resistor in connection Diagram?
- 3) What is the Command used to declare the inputs/outputs?
- 4) List out the types of LED's.

	JSPM University Pune Faculty of Science and Technology School of Computational Sciences TY BTECH CSE
Experiment No: 4	Date: / /
Aim : Controlling LED using push button with ESP32 (Digital Read)	

OBJECTIVE: The main objective of this experiment is,

- A. To read the Push button State Using Serial Monitor
- B. To turn ON an LED when a button is pressed, OFF when button is released.

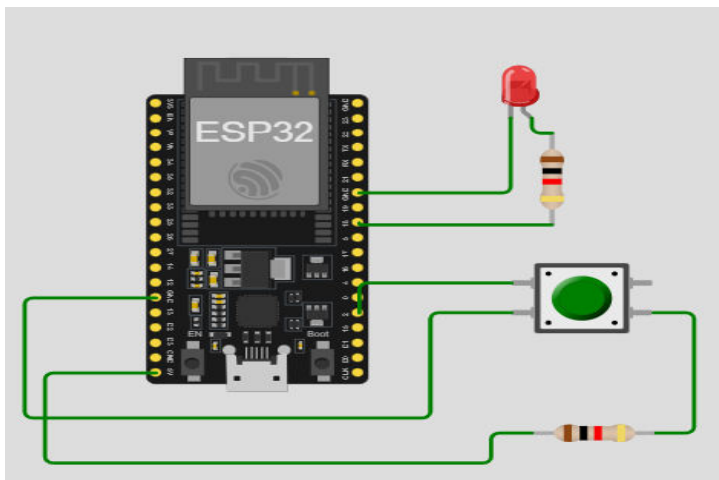
APPARATUS: Arduino UNO Board, ESP32 , LED , PUSH BUTTON , Jumper wires ,Breadboard

Description:

In this experiment we simply hook up 5 volts to one side of a button and to the other side of the button we connect pin 2. When you press the button it completes an electrical connection, pin 2 will “see” the 5 volts and if we digitalRead() at pin 2, it will report HIGH. During the times the button is not being pressed, pin 2 reports LOW.

We also have pin 2 connected to ground at all times through a resistor. This is because when we read values at pin 2, we want to get either a HIGH or a LOW reported. If pin 2 is not connected to ground, then when the button is not being pressed it becomes what is called a floating pin – it’s not connected to anything. Floating pins on the Arduino are fine for the most part – unless you are trying to record an input from them – then they are bad, and can give you spurious information.

INTERFACING DIAGRAM:



PROCEDURE:

- Connect one of the Arduino GND pins to one of the long power rails on the breadboard –this will be the ground rail.
- Connect the short leg of the LED to this same ground rail on the breadboard then connect the long leg to a row on the breadboard.
- Connect the 220-ohm resistor from pin 11 to the same row that you have the long leg of the LED attached.
- Place the pushbutton on the breadboard. Most buttons will straddle the center trench on the breadboard.
- Connect a jumper wire from the 5-volt pin to one side of the pushbutton.
- Connect a jumper wire from pin 7 to the other side of the pushbutton.
- Connect one side of the 10k resistor from the ground rail on the breadboard to the other side to the pushbutton – on the same side that pin 7 connects.

PROGRAM:

RESULTS:**CONCLUSION:****SAMPLE VIVA-VOCE QUESTIONS:**

- 1) How does push button work in ESP32?
- 2) What are the types of push button?
- 3) Does push button need resistor?
- 4) What are the applications of push button?
- 5)



JSPM University Pune
Faculty of Science and Technology
School of Computational Sciences
TY BTECH CSE

Experiment No: 5

Date: / /

Aim : Interfacing potentiometer with ESP32(Analog Read)

OBJECTIVE: The main objective of this experiment is,

- A. To read potentiometer data and print on the Serial Monitor
- B. To interface potentiometer with LED

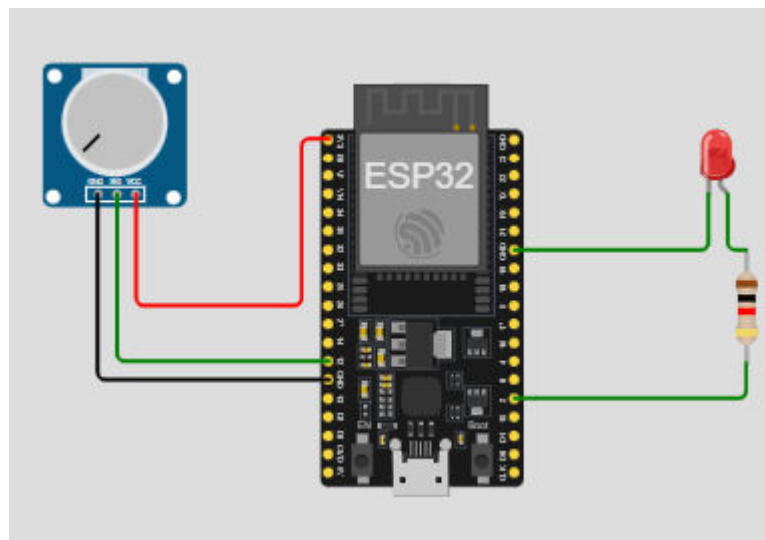
APPARATUS: ESP32 , LED , Potentiometer, Jumper wires , Breadboard.

DESCRIPTION:

The potentiometer is a device that is used to measure the voltage or electric potential. It provides a variable resistance when the shaft of the device is turned. The potentiometer is a three-terminal device. It has a rotating contact that acts as an adjustable voltage divider. The potentiometer structure consists of a sliding contact (called wiper), a resistive element, electrical terminals, and housing.

The sliding contact moves along the resistive element, while the housing consists of the wiper and the element.

INTERFACING DIAGRAM:



PROCEDURE:

- Connect potentiometer to ESP32 Board.
- Connect one terminal of potentiometer to GND
- Connect other terminal to 5V supply.
- Connect middle terminal to ANALOG pin

PROGRAM:

RESULTS:

CONCLUSION:

SAMPLE VIVA-VOCE QUESTIONS:

- 1) What is the purpose of potentiometer ?
- 2) What are the parts of potentiometer in ESP32?
- 3) What are the two main types of potentiometers?
- 4) How to control potentiometer with ESP32?



JSPM University Pune
Faculty of Science and Technology
School of Computational Sciences
TY B.Tech CSE- Sem-VI, 2025-26

Experiment No: 6

Date: / /

Aim : Smart lighting system using LDR and ESP32

OBJECTIVE: The main objective of this experiment is,

- A. To read LDR data and print on the Serial Monitor
- B. To interface LDR with LED.

APPARATUS: Arduino UNO Board, ESP32, LED, LDR, Jumper wires, Breadboard.

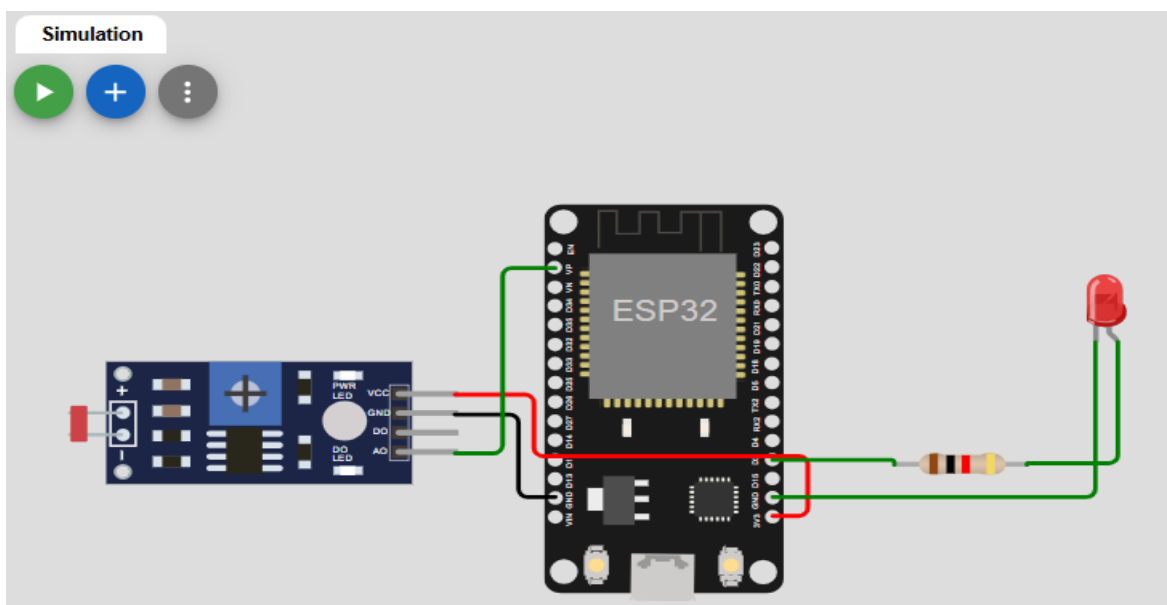
DESCRIPTION:

LDR (Light Dependent Resistor) as the name states is a special type of resistor that works on the photoconductivity principle means that resistance changes according to the intensity of light. Its resistance decreases with an increase in the intensity of light.

It is often used as a light sensor, light meter, Automatic street light, and in areas where we need to have light sensitivity. LDR is also known as a Light Sensor. LDR are usually available in 5mm, 8mm, 12mm, and 25mm dimensions

Two metal contacts are placed on both ends of the Zig-Zag shape these metal contacts help in creating a connection with the LDRs. Now, a transparent coating is applied on the top so that the zig- zag- shaped photosensitive material gets protected and as the coating is transparent the LDR will be able to capture light from the outer environment for its working.

INTERFACING DIAGRAM:



PROCEDURE:

- Connect one leg of the LDR to an analog input pin, for example, GPIO 34 (A0).
- Connect the other leg of the LDR to the 3.3V supply.
- You may need to add a resistor (e.g., 10k Ω) to create a voltage divider if your LDR doesn't have a built-in one or if your project requires it, connecting the resistor between the LDR and the ESP32's ground.

PROGRAM:

RESULTS:

CONCLUSION:

SAMPLE VIVA-VOCE QUESTIONS:

- 1) What is LDR?
- 2) Which type of sensor is LDR ?
- 3) Why LDR is used?



JSPM University Pune
Faculty of Science and Technology
School of Computational Sciences
TY B.Tech CSE- Sem-VI, 2025-26

Experiment No: 7

Date: / /

Aim : Motion Detection using PIR Sensor and ESP32

OBJECTIVE: The main objective of this experiment is,

A. To read the digital sensor value and display the sensor value in serial monitor.

APPARATUS: Arduino IDE, ESP32, Breadboard, Resistors, PIR sensor, connecting wires, supply source, Display

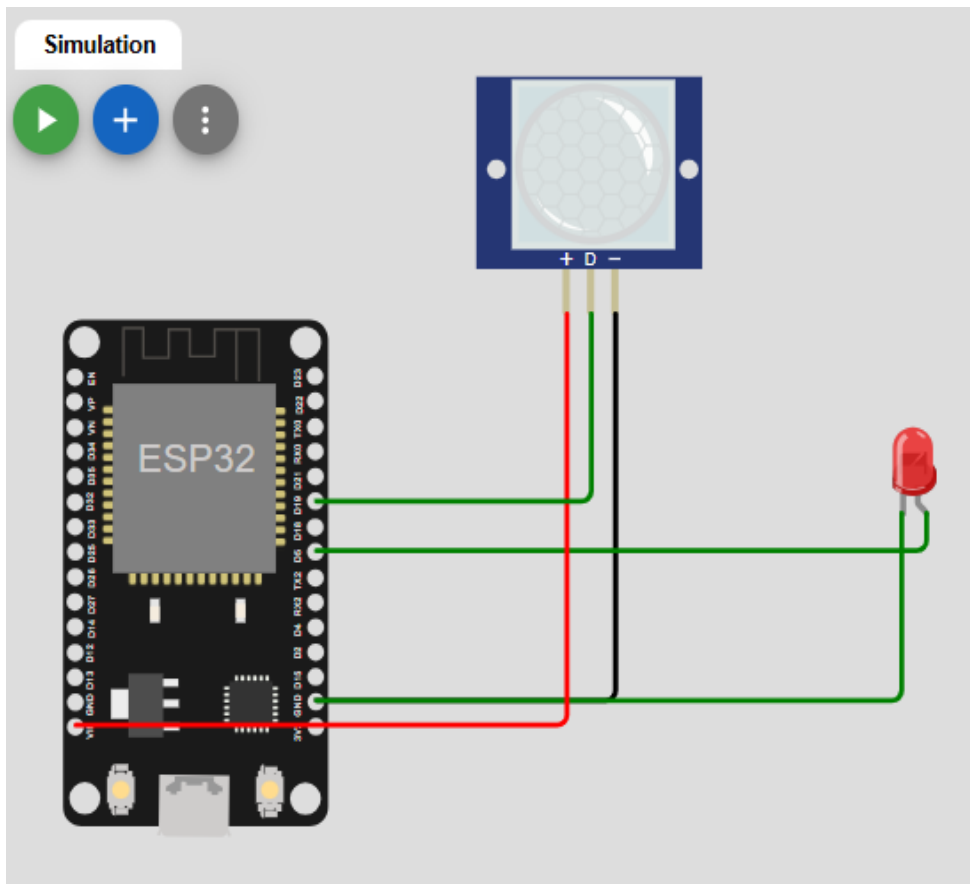
DESCRIPTION:

PIR stands for Passive Infrared Sensor. It is called passive sensor because it does not emit the IR rays. It only receives the IR rays. PIR sensor allows detecting or sensing the motion. It is small, Inexpensive and low power device. It is non-wearable device for that reason they are commonly found in appliances and gadgets used in homes or businesses. It is having 3 pin structure where,

- o Pin 1 is Vcc (+5v)
- o Pin 2 is Output
- o Pin 3 is ground

PIR sensor can detect animal/human movement in a requirement range. For Indoor passive infrared Detection distances range from 25 cm to 20 m. For Outdoor passive infrared, the detection distance ranges from 10 meters to 150 meters. It detects infrared radiation from the environment. Once there is infrared radiation from the human body particle with temperature, focusing on the optical system causes the device to generate a sudden electrical signal.

INTERFACING DIAGRAM:



PROCEDURE:

- Give the connections to the ESP32 board as shown in the connection diagram.
- Open Arduino IDE in the computer
- Create new file → New
- Type your program and Save it in appropriate location in your computer.
- Compile your program by clicking Verify option in the menu.
- Once the program compiled successfully, connect the Arduino board to the computer using USB cable.
- After connecting, go to Tools → Board → Select Arduino AVR board option.
- After selecting board, go to Tools → Port → Select Arduino UnoCOM port 3 (name may appear differently for other computers).

RESULTS:**ANALYSIS AND INTERFERENCE:****SAMPLE VIVA-VOCE QUESTIONS:**

- 1) What is the working principle of PIR sensor?
- 2) How do you interface a PIR sensor with ESP32?
- 3) What is the main function of PIR sensor?
- 4) What is the range of PIR sensor in Arduino?
- 5) State applications where PIR sensors can be used.



JSPM University Pune
Faculty of Science and Technology
School of Computational Sciences
TY B.Tech CSE- Sem-VI, 2025-26

Experiment No: 8

Date: / /

Aim: Temperature measurement using thermistor-LM35/DHT11 and ESP32.

OBJECTIVE: The main objective of this experiment is

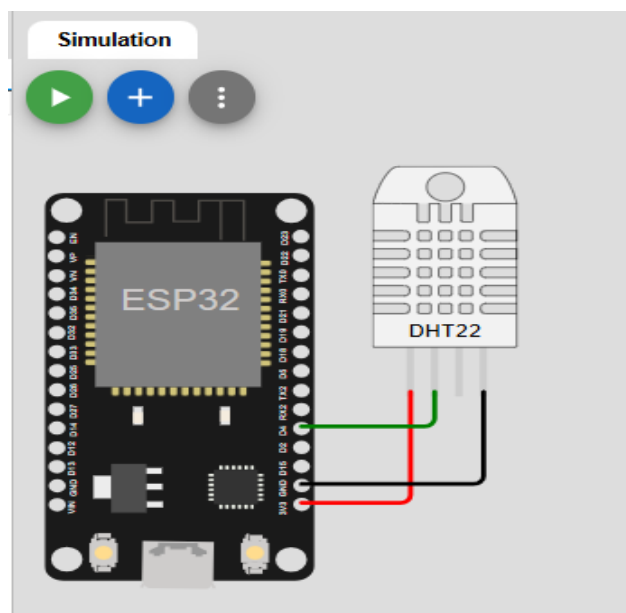
- A. To interface Temperature measurement using thermistor and ESP32

APPARATUS: ESP32, Arduino IDE, Breadboard, Resistors, Thermistor, connecting wires, supply source, Display.

DESCRIPTION:

A thermistor is a type of temperature sensor that changes its resistance with temperature variations. Specifically, we will be using an NTC (Negative Temperature Coefficient) thermistor, which has a higher resistance at lower temperatures and lower resistance at higher temperatures. By measuring the resistance of the thermistor, we can determine the temperature. The thermistor is connected to the Arduino UNO using two wires. One end of the thermistor is connected to the 5V pin of the Arduino, and the other end is connected to an analog input pin (e.g., A0). Additionally, a resistor (typically 10k ohms) is connected in parallel with the thermistor, with one end connected to the analog input pin and the other end connected to ground.

INTERFACING DIAGRAM:



PROCEDURE:

- Give the connections to the ESP32 board as shown in the connection diagram.
- Open Arduino IDE in the computer
- Create new file → New
- Type your program and Save it in appropriate location in your computer.
- Compile your program by clicking Verify option in the menu.
- Once the program compiled successfully, connect the esp32 to the computer using USB cable.
- After connecting, go to Tools →Board → Select Arduino AVR board option.
- After selecting board, go to Tools →Port → Select Arduino UnoCOM port 3 (name may appear differently for other computers).

PROGRAM:

RESULTS:**ANALYSIS AND INTERFERENCE:****SAMPLE VIVA-VOCE QUESTIONS:**

- 1) What is a thermistor?
- 2) How is the thermistor connected to the ESP32?
- 3) How can we convert the analog reading from the thermistor into temperature values?
- 4) How can we improve the accuracy of temperature measurements using the thermistor and ESP32?
- 5) What are some applications of temperature measurement using a thermistor and ESP32?



JSPM University Pune
Faculty of Science and Technology
School of Computational Sciences
TY B.Tech CSE- Sem-VI, 2025-26

Experiment No: 9

Date: / /

Aim: Distance measurement using SR-04 and ESP32.

OBJECTIVE: The main objective of this experiment is,

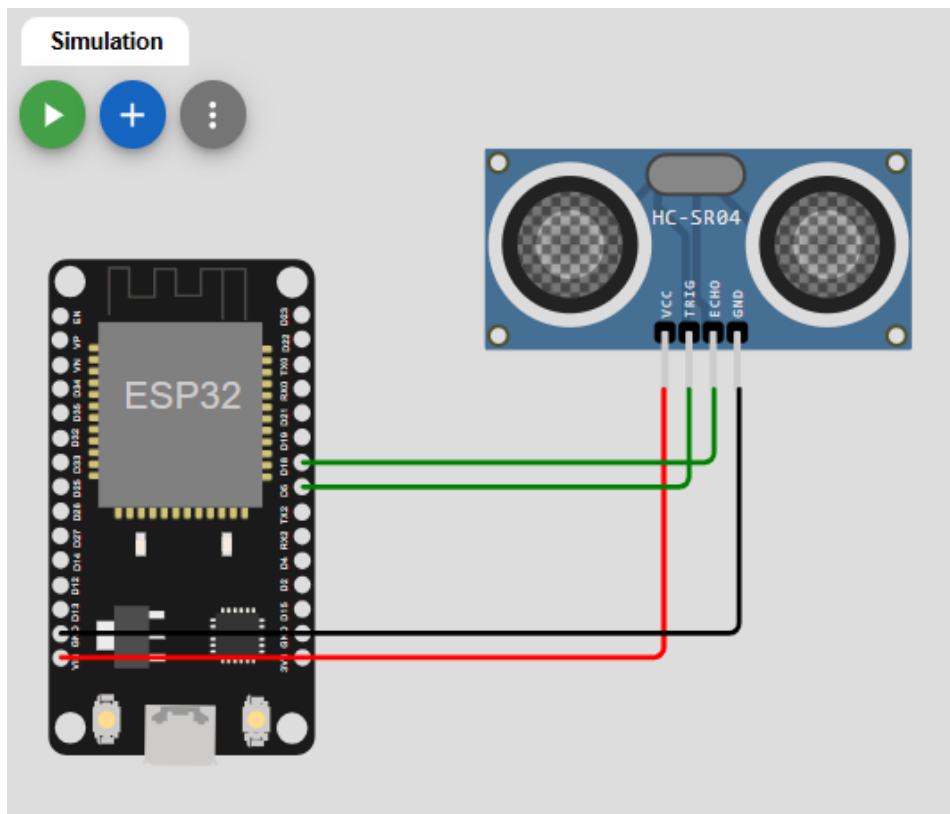
A.To Interface an Ultrasonic sensor to measure distance

APPARATUS: Arduino IDE, ESP32, Breadboard, Resistors, Ultrasonic sensor SR-04, connecting wires, supply source, Display.

DESCRIPTION:

Ultrasonic sensors work by emitting high-frequency sound waves and measuring the time it takes for the sound to reflect off an object and return to the sensor. This "time of flight" is then used with the known speed of sound to accurately calculate the distance to the object, similar to echolocation used by bats and dolphins.

INTERFACING DIAGRAM:



PROCEDURE:

- Give the connections to the ESP32 board as shown in the connection diagram.
- Open Arduino IDE in the computer
- Create new file → New
- Type your program and Save it in appropriate location in your computer.
- Compile your program by clicking Verify option in the menu.
- Once the program compiled successfully, connect the esp32 to the computer using USB cable.
- After connecting, go to Tools →Board → Select Arduino AVR board option.
- After selecting board, go to Tools →Port → Select Arduino UnoCOM port 3 (name may appear differently for other computers).

PROGRAM:

RESULTS:**ANALYSIS AND INTERFERENCE:****SAMPLE VIVA-VOCE QUESTIONS:**

- 1) What is the purpose of the SR-04 ultrasonic sensor in this experiment?
- 2) What is the formula used to calculate the distance using the SR-04 ultrasonicsensor?
- 3) What is the purpose of the trigPin and echoPin connections in the SR-04 ultrasonic sensor?
- 4) How does the SR-04 ultrasonic sensor measure distance?
- 5) How is the SR-04 ultrasonic sensor connected to the ESP32?
- 6) What is the maximum range of the SR-04 ultrasonic sensor?
- 7) How can the accuracy of distance measurement be improved with the SR-04 ultrasonic sensor?



JSPM University Pune
Faculty of Science and Technology
School of Computational Sciences
TY B.Tech CSE- Sem-VI, 2025-26

Experiment No: 10

Date: / /

Aim: Gas detection using MQ135/MQ2 and ESP32.

OBJECTIVE: The main objective of this experiment is,

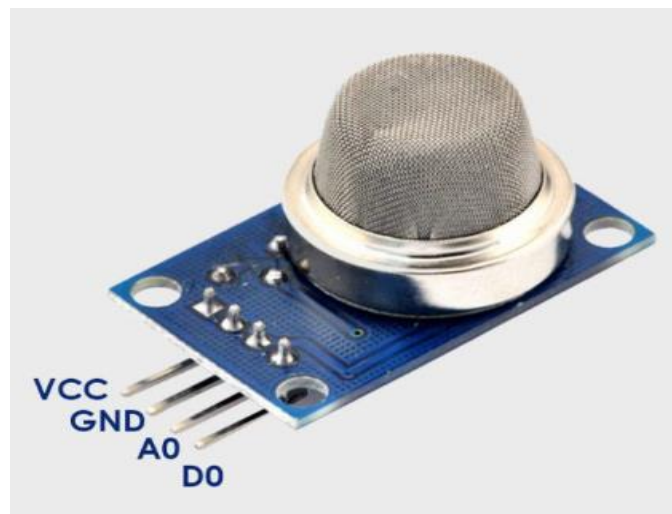
A. To interface Gas Detecting Sensor with ESP32.

APPARATUS: ESP32, Arduino IDE, Breadboard, Resistors, Gas Detecting sensor MQ-135/MQ-2, connecting wires, supply source, Display.

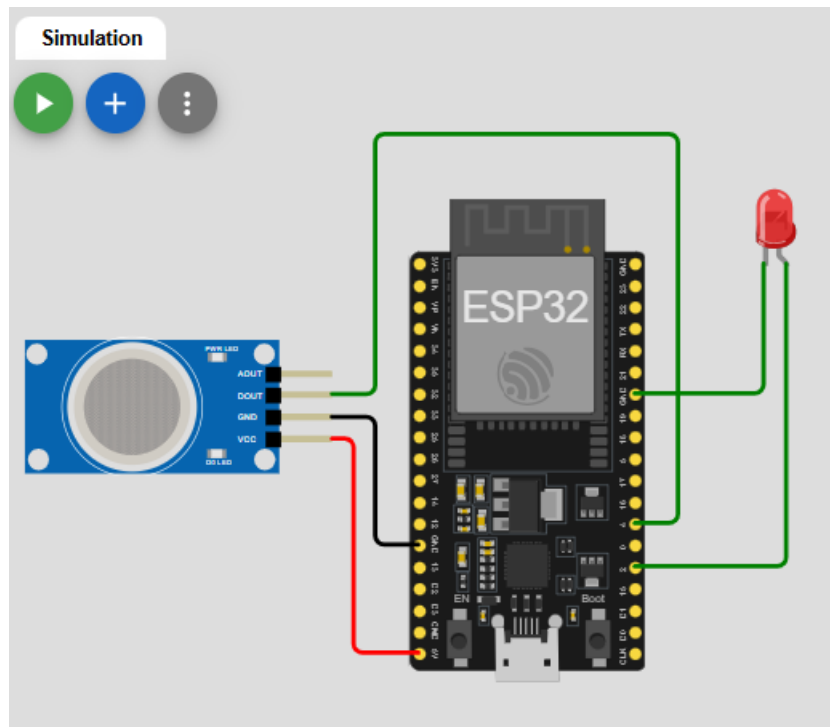
DESCRIPTION:

The MQ-2 sensor is a versatile gas sensor capable of detecting a wide range of gases including alcohol, carbon monoxide, hydrogen, isobutene, liquefied petroleum gas, methane, propane, and smoke. It is popular among beginners due to its low cost and easy-to-use features.

The MQ-2 sensor works on the principle of resistance changes in the presence of different gases. When the target gas comes in contact with the heated MOS (Metal Oxide Semiconductor) material, it undergoes oxidation or reduction reactions that change the resistance of the MOS material. It is noteworthy that the MQ2 gas sensor is capable of detecting multiple gases, but lacks the ability to differentiate between them. This is a common characteristic of most gas sensors. The sensor has a built-in potentiometer that allows you to adjust the sensor digital output (D0) threshold. When the concentration of gas in the air exceeds a certain threshold value, the resistance of the sensor changes.



INTERFACING DIAGRAM:



PROCEDURE:

- Give the connections to the ESP32 board as shown in the connection diagram.
- Open Arduino IDE in the computer
- Create new file → New
- Type your program and Save it in appropriate location in your computer.
- Compile your program by clicking Verify option in the menu.
- Once the program compiled successfully, connect the esp32 to the computer using USB cable.
- After connecting, go to Tools → Board → Select Arduino AVR board option.
- After selecting board, go to Tools → Port → Select Arduino UnoCOM port 3 (name may appear differently for other computers).

RESULTS:**ANALYSIS AND INTERFERENCE:****SAMPLE VIVA-VOCE QUESTIONS:**

- 1) What is the Principle of MQ2 Gas sensor?
- 2) What is the function of a gas sensor?
- 3) How do you interface a gas sensor?
- 4) What is the range of a gas sensor?
- 5) Is MQ2 Gas Sensor is Analog Sensor? If yes then why?



JSPM University Pune
Faculty of Science and Technology
School of Computational Sciences
TY B.Tech CSE- Sem-VI, 2025-26

Experiment No: 11

Date: / /

Aim: Servomotor (SG-90) control using ESP32.

OBJECTIVE: The main objective of this experiment is,

A. Servomotor (SG-90) control using ESP32.

APPARATUS: ESP32 Board, Servo Motor (SG-90), Jumper Wires, Breadboard

DESCRIPTION:

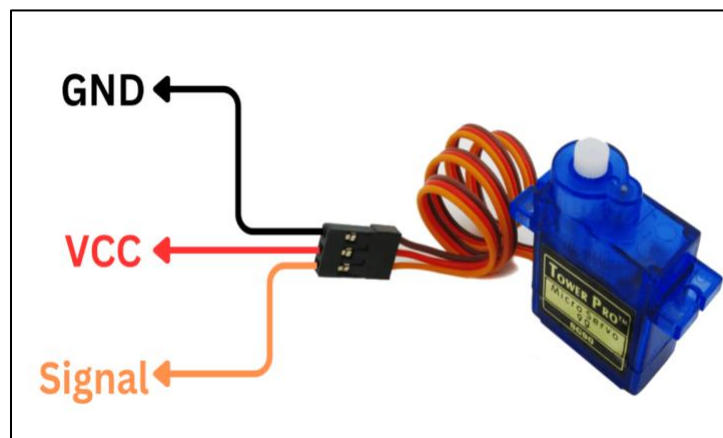
Servo motors are geared DC motors that have an integrated servomechanism with a feedback loop to allow precise positioning of the motor shaft. A high gear ratio allows a small servo to have an impressive torque rating.

Most servos are limited in rotation to either 180 or 270 degrees, with 180-degree servo motors being more common. There are specially modified servo motors that can rotate beyond 360-degrees, but we won't be working with those today.

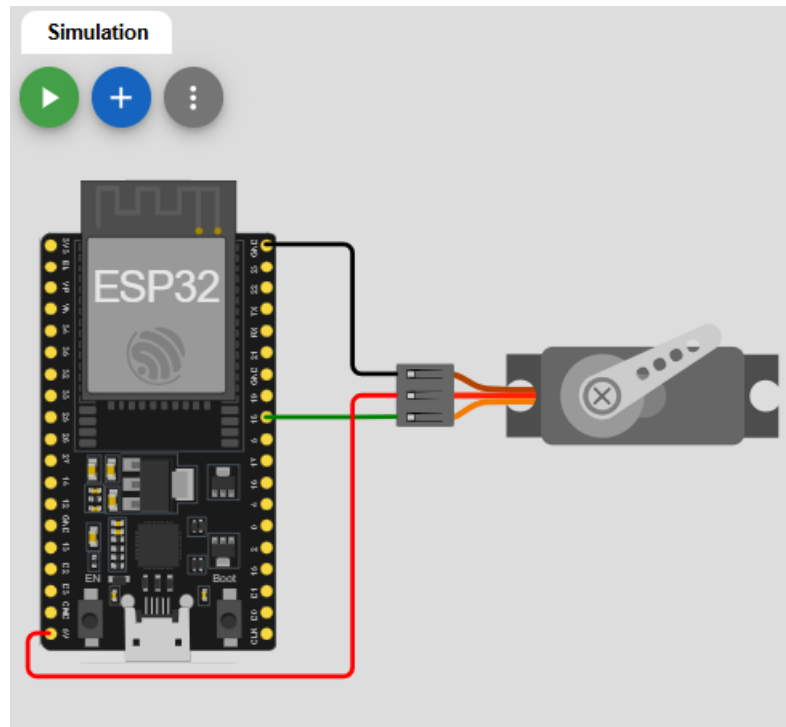
Servo motors come in a wide range of sizes and can be controlled either with an analog PWM signal or with a digital I/O signal.

A very common and inexpensive servo motor is the SG90. It's a small plastic-gear servo that has become a standard for experimenting and is also used in a lot of RC hobby applications. It has a "cousin", the MG90, which has similar specifications but uses metal gears.

The SG90, like most servo motors, can be powered from 5-6 volts, so servo motors are great for battery-powered applications.



INTERFACING DIAGRAM:



PROCEDURE:

- Give the connections to the ESP32 board as shown in the connection diagram.
- Open Arduino IDE in the computer
- Create new file → New
- Type your program and Save it in appropriate location in your computer.
- Compile your program by clicking Verify option in the menu.
- Once the program compiled successfully, connect the esp32 to the computer using USB cable.
- After connecting, go to Tools → Board → Select Arduino AVR board option.
- After selecting board, go to Tools→ Port →Select Arduino Uno COM port3 (name may appear differently for other computers).

Case No.	Case Name	Case Type	Case Status	Case Date	Case Location	Case Description	Case Details	Case Notes	Case Comments
1	John Doe	Case 1	Open	2023-01-01	New York	Case 1 Description	Case 1 Details	Case 1 Notes	Case 1 Comments
2	Jane Smith	Case 2	Closed	2023-01-02	California	Case 2 Description	Case 2 Details	Case 2 Notes	Case 2 Comments
3	Bob Johnson	Case 3	Pending	2023-01-03	Texas	Case 3 Description	Case 3 Details	Case 3 Notes	Case 3 Comments
4	Alice Brown	Case 4	Open	2023-01-04	Florida	Case 4 Description	Case 4 Details	Case 4 Notes	Case 4 Comments
5	Charlie Davis	Case 5	Closed	2023-01-05	Illinois	Case 5 Description	Case 5 Details	Case 5 Notes	Case 5 Comments
6	Eve White	Case 6	Pending	2023-01-06	Ohio	Case 6 Description	Case 6 Details	Case 6 Notes	Case 6 Comments
7	Frank Green	Case 7	Open	2023-01-07	Georgia	Case 7 Description	Case 7 Details	Case 7 Notes	Case 7 Comments
8	Grace Black	Case 8	Closed	2023-01-08	Arizona	Case 8 Description	Case 8 Details	Case 8 Notes	Case 8 Comments
9	Henry Blue	Case 9	Pending	2023-01-09	Colorado	Case 9 Description	Case 9 Details	Case 9 Notes	Case 9 Comments
10	Ivy Red	Case 10	Open	2023-01-10	Connecticut	Case 10 Description	Case 10 Details	Case 10 Notes	Case 10 Comments
11	Jack Purple	Case 11	Closed	2023-01-11	Delaware	Case 11 Description	Case 11 Details	Case 11 Notes	Case 11 Comments
12	Karen Yellow	Case 12	Pending	2023-01-12	Idaho	Case 12 Description	Case 12 Details	Case 12 Notes	Case 12 Comments
13	Liam Grey	Case 13	Open	2023-01-13	Indiana	Case 13 Description	Case 13 Details	Case 13 Notes	Case 13 Comments
14	Mia Silver	Case 14	Closed	2023-01-14	Iowa	Case 14 Description	Case 14 Details	Case 14 Notes	Case 14 Comments
15	Noah Gold	Case 15	Pending	2023-01-15	Kansas	Case 15 Description	Case 15 Details	Case 15 Notes	Case 15 Comments
16	Olivia Bronze	Case 16	Open	2023-01-16	Kentucky	Case 16 Description	Case 16 Details	Case 16 Notes	Case 16 Comments
17	Peter Copper	Case 17	Closed	2023-01-17	Louisiana	Case 17 Description	Case 17 Details	Case 17 Notes	Case 17 Comments
18	Quinn Iron	Case 18	Pending	2023-01-18	Maine	Case 18 Description	Case 18 Details	Case 18 Notes	Case 18 Comments
19	Rachel Steel	Case 19	Open	2023-01-19	Massachusetts	Case 19 Description	Case 19 Details	Case 19 Notes	Case 19 Comments
20	Sam Tin	Case 20	Closed	2023-01-20	Michigan	Case 20 Description	Case 20 Details	Case 20 Notes	Case 20 Comments
21	Tina Lead	Case 21	Pending	2023-01-21	Minnesota	Case 21 Description	Case 21 Details	Case 21 Notes	Case 21 Comments
22	Umar Zinc	Case 22	Open	2023-01-22	Mississippi	Case 22 Description	Case 22 Details	Case 22 Notes	Case 22 Comments
23	Victoria Nickel	Case 23	Closed	2023-01-23	Montana	Case 23 Description	Case 23 Details	Case 23 Notes	Case 23 Comments
24	Walter Silver	Case 24	Pending	2023-01-24	Nebraska	Case 24 Description	Case 24 Details	Case 24 Notes	Case 24 Comments
25	Xavier Gold	Case 25	Open	2023-01-25	Nevada	Case 25 Description	Case 25 Details	Case 25 Notes	Case 25 Comments
26	Yara Bronze	Case 26	Closed	2023-01-26	New Hampshire	Case 26 Description	Case 26 Details	Case 26 Notes	Case 26 Comments
27	Zoe Copper	Case 27	Pending	2023-01-27	New Jersey	Case 27 Description	Case 27 Details	Case 27 Notes	Case 27 Comments
28	Adam Iron	Case 28	Open	2023-01-28	New Mexico	Case 28 Description	Case 28 Details	Case 28 Notes	Case 28 Comments
29	Bella Steel	Case 29	Closed	2023-01-29	New York	Case 29 Description	Case 29 Details	Case 29 Notes	Case 29 Comments
30	Charlie Tin	Case 30	Pending	2023-01-30	North Carolina	Case 30 Description	Case 30 Details	Case 30 Notes	Case 30 Comments
31	Diana Lead	Case 31	Open	2023-01-31	North Dakota	Case 31 Description	Case 31 Details	Case 31 Notes	Case 31 Comments
32	Ethan Zinc	Case 32	Closed	2023-02-01	Ohio	Case 32 Description	Case 32 Details	Case 32 Notes	Case 32 Comments
33	Fiona Nickel	Case 33	Pending	2023-02-02	Oklahoma	Case 33 Description	Case 33 Details	Case 33 Notes	Case 33 Comments
34	George Silver	Case 34	Open	2023-02-03	Oregon	Case 34 Description	Case 34 Details	Case 34 Notes	Case 34 Comments
35	Hannah Gold	Case 35	Closed	2023-02-04	Pennsylvania	Case 35 Description	Case 35 Details	Case 35 Notes	Case 35 Comments
36	Ian Bronze	Case 36	Pending	2023-02-05	Rhode Island	Case 36 Description	Case 36 Details	Case 36 Notes	Case 36 Comments
37	Jessica Copper	Case 37	Open	2023-02-06	South Carolina	Case 37 Description	Case 37 Details	Case 37 Notes	Case 37 Comments
38	Kyle Iron	Case 38	Closed	2023-02-07	South Dakota	Case 38 Description	Case 38 Details	Case 38 Notes	Case 38 Comments
39	Laura Steel	Case 39	Pending	2023-02-08	Tennessee	Case 39 Description	Case 39 Details	Case 39 Notes	Case 39 Comments
40	Mason Tin	Case 40	Open	2023-02-09	Texas	Case 40 Description	Case 40 Details	Case 40 Notes	Case 40 Comments
41	Natalie Lead	Case 41	Closed	2023-02-10	Utah	Case 41 Description	Case 41 Details	Case 41 Notes	Case 41 Comments
42	Oscar Zinc	Case 42	Pending	2023-02-11	Vermont	Case 42 Description	Case 42 Details	Case 42 Notes	Case 42 Comments
43	Pamela Nickel	Case 43	Open	2023-02-12	Virginia	Case 43 Description			

RESULTS:**CONCLUSION:****SAMPLEVIVA-VOCEQUESTIONS:**

- 1) What is a servomotor?
- 2) How does a servo motor work?
- 3) How the SG-90 servo is motor connected to the ESP32?
- 4) How can we control the position of the servo motor using the ESP32?
- 5) What is the range of motion for the SG-90 serpv motor?